

StarryOS 自举编译与内核改进成果汇报

BigLab-B 最终汇报 | 杨凯森 | 2026-05

本工作以 StarryOS guest 内自举编译 StarryOS 为目标，形成了 OS 修复、测试验证、多核性能分析和复现文档。

12

已合入 OS PR

331s

8 核 guest self-build

2.87x

guest 端到端调优

6.67x

fork/exec/wait

成果总览：PR、性能与复现

本页汇总已完成的内核改进、StarryOS 自举编译结果和多核加速证据。

12 PR

#692 #693 #694 #695 #800
#842 #843 #844 #878 #879
#885 #926

331s

AArch64/HVF 8 核 guest 内完整
self-build PASS

951s -> 331s

guest 端到端调优 2.87x

20/11/5/3s

1000 次 fork/exec/wait, 8 路
6.67x

宿主对齐参考：85s -> 29s = 2.93x

同一 StarryOS AArch64/SMP8 kernel 配置与同 profile；只把
cargo/rustc 从 guest 移到 host。

结论：host 参考和 guest 对齐到同一构建任务；差别只剩运行在哪个 OS/FS/scheduler 上。

实验 1: AI 驱动的内核迭代框架

框架目标是把 AI 辅助开发接入真实 OS 工程流程，保证每次改动可验证、可回溯、可提交。



- 每日同步 TGOSKit dev 基线，PR 只基于 dev。
- 反馈链路超过 2h 就缩短：小测例、低日志、resume/cache、宿主预检。
- 确认 OS bug 后立刻转成 PR：root cause、最小修复、test-suite、CI。

内核改进成果：12 个已合入 PR

这些 PR 按 OS 行为边界拆分，覆盖进程线程、文件系统、网络 ABI、SMP 同步和调度前进性。

进程 / 线程 / futex

#692 robust futex cleanup
#693 vfork parent blocking
#878 teardown context

文件系统 / 设备

#695 rsex4 inode bitmap
#800 device full transfer
#844 tmpfs rename exec

SMP / 同步 / 调度

#842 CPU topology
#879 raw mutex handoff
#885 syscall entry snapshot
#926 SMP wakeup progress

网络 ABI

#694 IPv4-mapped IPv6 socket
保持 IPv6 用户态语义，内部复用 IPv4 backend。

验证标准

每个 PR
均包含问题根因、修复范围、测试用例
和 CI 或本地复现证据。

重点 PR 一：vfork/clone 语义修复

该 PR 恢复 CLONE_VFORK 父进程等待子进程 exec/exit 的 Linux ABI，影响 posix_spawn 与 shell 子进程路径。

问题

- CLONE_VFORK 父进程必须等到子进程 exec 或 exit。
- 把等待条件错误收窄到 `stack == 0`，会破坏 BusyBox、shell、timeout 依赖的父子同步顺序。

修复与测例

- 所有 CLONE_VFORK 子进程都建立并等待 `vfork_done`。
- 测例：test-vfork。
- 影响范围：BusyBox / shell / cargo 子进程创建路径。

成果：修复进程创建 ABI 语义，使 BusyBox、shell 和 cargo 子进程创建路径具备更稳定的行为基础。

重点 PR 二：futex 与线程退出清理

该组修复提升了线程退出路径对用户态坏地址和 pending futex 状态的容错能力。

问题

- robust-list bad head 可能来自用户态坏指针。
- pending futex 和普通 entry 清理混在一起，会污染退出路径。

修复与测例

- 坏 entry 容错清理，pending 单独处理。
- 测例：test-futex-robust-list。
- 关联：teardown/context usercopy 修复退出路径。

成果：退出清理路径可以安全处理用户态异常输入，避免单个坏 robust-list 破坏进程回收流程。

重点 PR 三：文件系统与 VFS 正确性

这些修复面向真实应用的文件系统访问模式，补齐 inode 分配、rename/exec 和设备传输语义。

#695 rsex4 inode bitmap

未初始化 inode bitmap
不能直接跳过整个 block
group；应初始化后继续分配。

#844 tmpfs rename exec

rename/unlink 与 exec/open
组合不能留下不一致目录状态。

#800 device transfer

设备读写需要支持完整
transfer，避免用户态工具拿到短读
写。

成果：文件系统层面对真实应用的目录项、inode 和设备 I/O 行为更加接近 Linux 预期。

重点 PR 四：SMP 同步与调度前进性

多核自举编译暴露了短任务、等待唤醒和远端 runqueue 上的前进性问题。

#842 CPU topology

给用户态暴露正确 CPU 拓扑，避免多核环境下配置和调度判断失真。

#879 RawMutex handoff

修复同步原语 handoff 边界，降低锁竞争下的错误唤醒风险。

#926 wakeup progress

wait queue / remote runqueue wakeup 需要保证前进性。

- 验证：fork/exec/wait 1000 次在 j1/j2/j4/j8 下为 20/11/5/3s。
- 分析：完整 cargo 的剩余瓶颈集中在 T_wait、T_smp、T_fs 和串行尾段。

实验 4: StarryOS guest 内自举编译

实验目标是在 StarryOS userland 中运行 cargo build, 完成 StarryOS 自身构建并输出 PASS marker。



- 宿主对齐参考：同 StarryOS kernel 配置、同 target/profile, host cargo j1=85s, j8=29s。
- guest 最优：AArch64/HVF 8 核 StarryOS guest, 完整 self-build 331s。
- 判据：进入 userland、找到 rootfs 内 StarryOS ELF、打印 PASS、无 panic/trap/FATAL/error。

多核编译性能结果

本页拆分端到端、编译并行、链接/串行尾段三种加速比，避免混用分母。

951s

最慢 guest baseline

642s

默认 8 核 guest

331s

当前最快 guest

29s

host cargo j8

85s

host cargo j1

端到端真正收益: $951s / 331s = 2.87x$

guest 编译并行: $422s / 341s = 1.24x$

host 对齐参考: $85s / 29s = 2.93x$

链接/串行尾段: $642s / 427s = 1.50x$

注: host 与 guest 编译同一 AArch64/SMP8 StarryOS kernel; host 不进入 guest, 链接项不是独立 link-only benchmark。

结论: 完整 guest self-build 已完成; 并行收益受 Cargo critical path、guest OS/FS/SMP 和串行尾段共同限制。

性能模型与测试细项

每个瓶颈都用接近 Cargo 行为的微基准拆开验证，而不是只给粗略归因。

$$T_{\text{build}}(N) = T_{\text{std}}/\text{cache} + T_{\text{serial}} + T_{\text{parallel}}/N + T_{\text{fs}}(N) + T_{\text{smp}}(N) + T_{\text{wait}}(N)$$

进程链路

fork/exec/wait 1000
次: 20/11/5/3s, 8 路
6.67x。模拟 cargo 反复
spawn rustc。

短任务 wave

24 个
build-script: 27/22/27/32
/32s。2
路最快，过并行会被
OS/FS/SMP 成本吃掉。

FS metadata

rsext4 fileio
jobs=8: 47.5s→14.5s。模拟
target 小文件
create/write/rename/unlink
。

宿主/QEMU参考

host 同构建任务
85/29=2.93x; guest
jobs-only
422/341=1.24x。raw CPU
MTTCG 3.17x 不作 RISC-V
correctness。

final link timestamp: artifact tail 约 2s, starry-kernel 后段约 17–20s; 331s 版本主瓶颈不是纯链接。

对齐对比：为什么 host 能 2.93x, guest 只有 1.24x

这页把同一 StarryOS kernel 构建任务的 host 参考和 guest jobs-only 放在一起，说明差距来自哪里。

85s → 29s

host 对齐参考, 2.93x

422s → 341s

guest jobs-only, 1.24x

951s → 331s

guest 端到端调优, 2.87x

20/11/5/3s

进程短链路, 6.67x

- host 对齐参考：同源码、同 AArch64/SMP8 kernel config、同 release profile；只是不进入 guest，不承担 StarryOS syscall、VFS、scheduler。
- guest jobs-only：同 profile 下只改 jobs，真实承担 fork/exec/wait、pipe、FS metadata、runqueue、锁竞争和虚拟化边界。
- 结论：并行空间存在；没有线性加速不是因为多核无效，而是 full cargo 把 OS 短任务成本和串行尾段叠在一起。

现场讲法：host 是可并行上界；guest 是 OS 真实承载能力；两者差距就是本实验定位和修复的对象。

成因拆解：Cargo、OS、QEMU 分别承担什么

最后把瓶颈按归属拆清楚，避免把所有慢都归因给 QEMU 或 Cargo。

Cargo / Rust 部分

build-std、proc-macro、build.rs、create critical path 和尾部 codegen/link 形成串行段；`jobs=12`、`-Z threads=4` 都已证明过并行会反吃。

StarryOS 内核部分

进程等待/唤醒、runqueue 前进性、CPU topology、VFS metadata 和 ext4 sync 策略会影响短任务吞吐；#842/#926 和 rsext4 短测例对应这部分。

QEMU/HVF 部分

AArch64/HVF 去掉了 RISC-V TCG 翻译主开销，但仍有虚拟块设备、中断、VM exit 和 guest/host 边界；RISC-V MTTCG 只作 raw CPU 参考。

最 solid 的结论：完整 self-build 已跑通；OS 短链路能扩展到 6.67x；full cargo 的剩余瓶颈被拆成可复现、可继续提 PR 的具体问题。

复现材料与答辩支撑

所有关键结果均保留复现命令、运行日志、PASS marker、PR 记录和最终报告材料。

结果文档

showtime-2/final/README.md
包含性能
setting、日志路径和复现入口。

运行日志

showtime-2/logs/*
live-demo/out/*.log
包含 PASS
marker、elapsed、arch、smp、jobs
等字段。

PR 记录

success-pr/PR-TRACKING.md
记录已合入
PR、验证证据和重点修复说明。

总结：本工作完成了 StarryOS 自举编译验证，并将真实 workload 暴露的问题沉淀为可合入的 OS 修复与测例。